



Casa abierta al tiempo

Universidad Autónoma Metropolitana

Unidad Iztapalapa

División de Ciencias Básicas e Ingeniería

Licenciatura en Computación

Proyecto Terminal de Investigación 1

Informe — Semana 8

Detección Inteligente de Paráfrasis mediante Modelos de Lenguaje de Gran Escala (LLM).

Mejora del sistema de doble modo: diagnóstico de los sistemas previos, refuerzo del Modo 1 y evaluación honesta sobre el benchmark robusto

Asesor: Dr. Benjamin Moreno Montiel

Alumno: Juan Antonio Soria Rodríguez

Matrícula: 2203041232

Grupo: CK06

Fecha: Junio 2026

Resumen

Este informe documenta la **octava semana** del proyecto, dedicada a **mejorar** el sistema de detección de paráfrasis y copia de doble modo construido en la semana 7 (sistema v4), sin cambiar el objetivo del proyecto: una *plataforma de doble modo* que (1) compara dos textos y dictamina si son paráfrasis (Modo 1) y (2) compara un fragmento o una tesis completa contra el corpus TESIAMI/CBI para encontrar paráfrasis, similitudes o copia literal (Modo 2). El trabajo parte de una premisa autocrítica heredada de las auditorías: el **Modo 1** es el componente más frágil, porque su evaluación canónica descansa en muy pocos pares de paráfrasis semántica realmente difíciles; y aunque el corpus CBI es *original* (no hay plagio entre tesis), el Modo 2 **sí** detecta similitudes reales —solapamiento temático y terminológico entre áreas afines (p.ej. matemáticas y química) y copia casi literal de fuentes compartidas, como se documentó en el caso de la tesis de extracción de plata (semana 7)—. Antes de re-entrenar o medir nada, la semana 8 propone (i) un **estudio diagnóstico** de los sistemas previos (v2, v3, v4) para identificar qué aporta cada uno y qué conviene rescatar; (ii) el **refuerzo del Modo 1**; y (iii) una **evaluación honesta sobre el benchmark robusto** (`test_v4`, 109 pares, 57 positivos reales), siguiendo la recomendación de la auditoría de no medir sobre el test pequeño de 38 pares. Se mantienen las reglas de integridad: doble número etiquetado (honesto/referencia), todo conjunto con su sesgo declarado y cada resultado trazado a su archivo.

Índice

1. De dónde partimos: la base de la semana 7	3
1.1. El sistema v4 (entregable vigente)	3
1.2. Los dos modos del sistema	3
1.3. Lo que ya quedó demostrado (y manda)	3
1.4. Estado de integridad numérica (heredado)	4
2. Requerimientos de la semana 8: mejorar el sistema v4	4
2.1. R8.1 — IA explicativa	4
2.2. R8.2 — Idioma del corpus y validación del modelo multilingüe	4
2.3. R8.3 — Arquitecturas de búsqueda: cobertura 100 % y rapidez	5
2.4. R8.4–R8.7 — Mejora del sistema, evaluación y no-regresión	5
3. Plan de trabajo: investigación · experimentación · diseño · implementación	6
4. I-1 — Idioma del corpus CBI	6
4.1. Objetivo (R8.2)	6
4.2. Método	7
4.3. Resultados	7
4.4. Conclusión: ¿justifica el modelo multilingüe?	8
5. I-2 — Arquitecturas de búsqueda vectorial (estado del arte)	8
5.1. El problema a resolver (R8.3)	8
5.2. Familias de índices (FAISS)	8
5.3. Comparación cualitativa (a priori)	9
5.4. Qué medirá E-1 (protocolo)	9
6. I-3 — IA explicativa (estado del arte y opciones)	9
6.1. El problema a resolver (R8.1)	9
6.2. Familias de métodos de explicabilidad	10

6.3.	Comparación de métodos	10
6.4.	Propuesta: explicabilidad por capas (alimenta D-2)	11
7.	E-1 — Benchmark de arquitecturas de búsqueda	11
7.1.	Protocolo	11
7.2.	Resultados	11
7.3.	Análisis	12
8.	E-2 — Diagnóstico de los sistemas previos (v2/v3/v4)	13
8.1.	Inventario de componentes (R8.4)	13
8.2.	Prueba empírica de complementariedad	13
9.	E-4 — Refuerzo del Modo 1: estudio de errores y datos	14
9.1.	Estudio antes de evaluar (análisis de errores)	14
9.2.	Datos: minado de negativos difíciles reales	14
9.3.	Datos: candidatos a positivos difíciles (verificación humana)	15
9.4.	Diseño del experimento E-4	15
9.5.	Resultados (Colab, 3 semillas, sobre <code>test_v4</code>)	15
10.P-1	— Integración de HNSW en el sistema	16
10.1.	Cambio realizado	16
10.2.	Verificación funcional	16
10.3.	Impacto real: el Modo 2 de tesis completa	16
11.D-2 / P-2	— IA explicativa anclada al texto	17
11.1.	Diseño (D-2)	17
11.2.	Implementación (P-2)	17
11.3.	Verificación	17
12.P-3	— Búsqueda híbrida (BM25 + HNSW, RRF)	18
12.1.	Motivación (de E-2)	18
12.2.	Implementación	18
12.3.	Verificación	18
13.	Mejoras y validaciones del sistema v5	19
13.1.	El techo del Modo 1 (tres resultados negativos triangulados)	19
13.2.	Métricas honestas: k-fold, IC95 % y estratificación por dificultad	19
13.3.	Validación de la búsqueda híbrida (sí mejora)	20
13.4.	Híbrido extendido al reporte de tesis completa	20
13.5.	Reporte honesto del Modo 2 (filtro de boilerplate)	20
13.6.	Usabilidad: reporte profesional	20
13.7.	R7: Flags Panel de integridad (adoptado de iThenticate 2.0)	21
13.8.	R7 (bis): adopción de Compilatio (línea de tiempo, filtro y umbrales)	21
14.	Conclusiones de la semana 8	21
14.1.	Lo que mejoró el sistema	21
14.2.	Lo que NO mejoró (resultado negativo honesto)	22
14.3.	Estado del entregable	22
14.4.	Trabajo futuro	22

1. De dónde partimos: la base de la semana 7

La semana 8 no arranca de cero: **mejora** un sistema que ya existe, corre y está documentado. Esta sección resume esa base —el sistema v4— para fijar con precisión el punto de partida y el alcance de la mejora.

1.1. El sistema v4 (entregable vigente)

El sistema v4 (`semana_7/sistema_v4/`) es un detector de paráfrasis y copia **unificado de doble modo**, servido como aplicación web (`./run.sh web → localhost:8060`) e instalable como PWA. Sus componentes:

- **Recuperación.** Un *bi-encoder* (MiniLM multilingüe, 384 dim) genera embeddings y consulta un índice **FAISS del corpus CBI al 100 % (2 284 185 vectores, 4 954 tesis)**, con conmutación automática $\text{IVFPQ} \rightarrow \text{IVFFlat} \rightarrow \text{plano}$ según RAM y precisión.
- **Clasificación.** Un *cross-encoder* seleccionable: **XLM-R v4** (calibrado, $T = 5.15$), **BETO v3 o ensamble** (soft-voting).
- **Reglas de decisión.** Combina probabilidad del clasificador, similitud semántica (MiniLM) y solapamiento léxico (BLEU-2) en **6 tipos** con severidad 0–3 (duplicado exacto, copia casi literal, paráfrasis, alta similitud, mismo tema, no relacionados).
- **Capa LLM (opcional).** Árbitro (Groq Llama 3.3 70B) en la zona de duda $[0.35, 0.65]$ y generación de explicaciones e informe ejecutivo.
- **Modo 2 — tesis completa.** Fragmenta, asigna sección, busca contra el corpus (excluyendo la propia tesis) y produce un *reporte de similitud* con porcentaje global, por sección, fuentes principales, zonas de calor y evidencia.

1.2. Los dos modos del sistema

Modo 1. Dados dos textos $\rightarrow$ ¿son paráfrasis? Es el *clasificador de pares*. Métricas: probabilidad del cross-encoder, similitud semántica y BLEU-2.

Modo 2. Dado un fragmento o una tesis completa $\rightarrow$ se compara contra el corpus para hallar paráfrasis, similitudes o texto literal copiado, indicando *qué tesis, qué similitud y en qué porcentaje*.

1.3. Lo que ya quedó demostrado (y manda)

- **El dato humano vence al sintético.** El *fine-tuning* sobre datos generados (LLM, back-translation) *degrada* el modelo: aprende el estilo del generador, no la relación de paráfrasis. Confirmado en v4–v7 y, en semana 7, de nuevo en E-5 (gold-solo $F_1 = 0.812 > \text{gold+silver } 0.786$ sobre `test_v4`).
- **Empate dentro del ruido.** BETO v3 y XLM-R v4 calibrado quedan estadísticamente empatados (AUROC 0.98–0.99); el aporte real es **eficiencia de datos**.
- **El corpus CBI es original** —no hay plagio parafraseado *entre tesis*—, pero el Modo 2 sí encuentra similitudes legítimas: *boilerplate* institucional, bibliografía compartida casi literal y solapamiento temático/terminológico entre áreas afines. El caso de la tesis de **extracción de plata** (química) ilustra cómo, al comparar un documento contra el corpus, aparecen coincidencias reales que el sistema clasifica correctamente.

1.4. Estado de integridad numérica (heredado)

Tras la última revisión, el sistema, el informe y la bitácora reportan el mismo **doble número etiquetado**: el *honesto* (test robusto/realista) como titular y el de *referencia* (test pequeño o balanceado) como contexto secundario.

Modelo	Honesto (titular)	Referencia
XLM-R v4	$F_1 = 0.812$, AUROC 0.839 — <code>test_v4</code> (109, 57 pos)	0.941 — <code>test_v3_limpio</code> (38)
BETO v3	$F_1 = 0.842$, AUROC 0.989 — <code>test_v3_limpio</code>	0.957 — <code>test_v4_clean</code> (63, bal.)
Ensamble	$F_1 = 0.889$, AUROC 0.996 — <code>test_v3_limpio</code>	0.971 — <code>test_v4_clean</code> (bal.)

Cuadro 1: Punto de partida: métricas honestas heredadas de la semana 7.

Punto de partida de la semana 8

Tenemos un sistema de doble modo **funcional**, **indexado al 100 %** y **honesto en sus números**, con dos debilidades conocidas: (a) el **Modo 1** se evalúa sobre muy pocos positivos difíciles reales, y (b) la mejora del clasificador está **empatada dentro del ruido**. La semana 8 ataca exactamente eso.

2. Requerimientos de la semana 8: mejorar el sistema v4

El objetivo de la semana 8 es **mejorar el sistema construido en la semana 7**, no sustituirlo. La meta del proyecto no cambia: sigue siendo la plataforma de doble modo, mutación del enfoque clásico de Luna (2022) hacia un paradigma basado en *transformers* y LLM. A partir del diagnóstico de la Sección 1 y de las recomendaciones de las auditorías, se definen los siguientes requerimientos.

Los requerimientos se agrupan en dos bloques: los **solicitados explícitamente por el asesor** para esta semana y los de **mejora/pendientes** que arrastramos.

2.1. R8.1 — IA explicativa

El sistema ya genera explicaciones por reglas y un árbitro/explicador LLM (semana 7). El asesor pide **profundizar** la explicabilidad: que la salida no solo dé un veredicto y un porcentaje, sino que *justifique* por qué —resaltando los fragmentos/frases que disparan la coincidencia, distinguiendo paráfrasis real de bibliografía compartida, terminología común o *boilerplate*, y produciendo un razonamiento legible—. Se investigarán métodos (atribución por atención, resaltado de *spans*, *rationales* con LLM) y se implementará la opción más adecuada en ambos modos.

2.2. R8.2 — Idioma del corpus y validación del modelo multilingüe

La semana 7 adoptó un *backbone* multilingüe (XLM-RoBERTa). El asesor pide **sustentar** esa decisión con datos: medir, sobre las 4954 tesis del corpus CBI, **qué porcentaje del texto está en español y qué porcentaje en otro idioma** (típicamente inglés en resúmenes, *abstracts*, citas y terminología), con histograma y tabla por idioma. Con ese dato se discute si el modelo multilingüe nos conviene o si un modelo monolingüe sería suficiente.

ID	Tema	Qué exige
<i>Solicitados por el asesor (semana 8)</i>		
R8.1	IA explicativa	Investigar e implementar IA explicativa para que la salida del sistema sea más explicativa (por qué dos textos se parecen, qué evidencia lo sostiene).
R8.2	Idioma del corpus + XLM-R multilingüe	Medir qué porcentaje del corpus está en otro idioma (histograma/tabla) y verificar si el modelo multilingüe (XLM-R/RoBERTa) presentado nos conviene a la luz de ese dato.
R8.3	Arquitecturas de búsqueda	La búsqueda sobre el índice al 100 % es algo lenta . Explorar nuevas arquitecturas de búsqueda vectorial, hacer pruebas con las nuevas y las ya usadas, y elegir la mejor candidata con métricas: <i>recall</i> , medición de tiempo , RAM. Meta: cubrir el 100 % de búsqueda y que sea rápida .
<i>Mejora del sistema / pendientes</i>		
R8.4	Estudio diagnóstico de sistemas previos	Comparar v2, v3 y v4: qué aporta cada uno y qué conviene rescatar para el sistema mejorado (BM25, FAISS de oraciones, reglas de tipo, manejo de <i>boiler-plate</i>).
R8.5	Refuerzo del Modo 1	Mejorar el clasificador de pares (eslabón débil): ampliar positivos difíciles reales y re-ajustar, midiendo sobre el benchmark robusto.
R8.6	Evaluación honesta sobre test_v4	Medir sobre test_v4 (109, 57 pos), no sobre los 38, con un estudio previo que oriente la mejora (análisis de errores) antes de evaluar.
R8.7	Integración y no-regresión	Integrar las mejoras en el sistema mejorado (v5) con doble número etiquetado, trazabilidad y pruebas de funcionamiento , sin romper lo que el v4 ya hace bien.

Cuadro 2: Requerimientos de la semana 8 (asesor + mejora). Se refinan con cada instrucción.

2.3. R8.3 — Arquitecturas de búsqueda: cobertura 100 % y rapidez

Hoy el índice cubre el 100 % del corpus (2.28M vectores) con IndexFlatIP (exacto pero lento) y variantes IVF/IVFPQ (rápidas pero aproximadas). El asesor pide **explorar nuevas arquitecturas** (p.ej. HNSW, IVF-HNSW, PQ optimizado) y **comparar** todas —nuevas y actuales— con un *benchmark* formal: **tiempo de consulta** (mediana, p95), *recall@k* contra la búsqueda exacta, uso de RAM y tamaño en disco. El objetivo es elegir, con evidencia, la arquitectura que mantenga **cobertura del 100 %** de la búsqueda con la **menor latencia**.

2.4. R8.4–R8.7 — Mejora del sistema, evaluación y no-regresión

En paralelo se ejecutan: el **diagnóstico de los sistemas previos** (qué rescatar), el **refuerzo del Modo 1** con datos difíciles reales, la **evaluación honesta sobre test_v4** precedida de análisis de errores, y la **integración** de todo en el sistema mejorado (v5) con pruebas de funcionamiento, manteniendo el encuadre honesto.

3. Plan de trabajo: investigación · experimentación · diseño · implementación

Para cubrir los requerimientos de forma ordenada se adopta el mismo esquema de cuatro fases del proyecto. Cada tarea produce un **entregable verificable** (script, dato, gráfica, sección de este informe) y se documenta a detalle en `semana_8/DOCUMENTACION.md`.

Fase	Tarea	Entregable	Cubre
<i>Investigación (I)</i>			
I-1	Análisis de idioma del corpus	% por idioma + histograma	R8.2
I-2	Estado del arte: arquitecturas de búsqueda vectorial	tabla comparativa	R8.3
I-3	Métodos de IA explicativa (atribución/ <i>spans</i> / <i>rationales</i>)	nota técnica	R8.1
<i>Experimentación (E)</i>			
E-1	<i>Benchmark</i> de arquitecturas de búsqueda (latencia/recall/RAM)	tabla + gráficas	R8.3
E-2	Diagnóstico comparativo v2/v3/v4	informe de componentes	R8.4
E-3	Validación del modelo multilingüe vs idioma medido	métricas por idioma	R8.2
E-4	Refuerzo del Modo 1 + evaluación sobre <code>test_v4</code>	modelo + métricas	R8.5/R8.6
<i>Diseño (D)</i>			
D-1	Selección de la arquitectura de búsqueda ganadora	decisión justificada	R8.3
D-2	Diseño de la capa de IA explicativa	especificación	R8.1
D-3	Diseño del sistema mejorado (v5) sin romper v4	arquitectura	R8.7
<i>Implementación (P)</i>			
P-1	Integrar la búsqueda rápida al sistema	código + prueba	R8.3
P-2	Implementar IA explicativa en Modo 1 y Modo 2	código + prueba	R8.1
P-3	Integrar el Modo 1 reforzado	código + prueba	R8.5
P-4	Pruebas de funcionamiento <i>end-to-end</i> del sistema v5	reporte de pruebas	R8.7

Cuadro 3: Plan de cuatro fases de la semana 8. Orden sugerido; se ajusta según resultados.

Orden de ejecución sugerido. Se arranca por las tareas de *bajo costo y alto valor* que no requieren GPU y responden directamente a una pregunta del asesor: **I-1** (idioma del corpus) y **E-1** (*benchmark* de búsqueda sobre el índice ya construido). Con esos datos se decide el diseño de la búsqueda (D-1) y, en paralelo, se investiga la IA explicativa (I-3/D-2). Los entrenamientos (E-4) y la integración final (P-1–P-4) se abordan después, con Colab/GPU cuando haga falta.

Cómo se documentará

Todo lo que se haga —cómo se programa, qué datos entran, qué métricas salen— se documenta a detalle aquí, con **gráficas, tablas e histogramas**, profundizando en cada decisión. Este informe crece tarea a tarea; la bitácora viva está en `semana_8/DOCUMENTACION.md`.

4. I-1 — Idioma del corpus CBI

4.1. Objetivo (R8.2)

El asesor pidió **medir qué porcentaje del corpus está en español y qué porcentaje en otro idioma**, para sustentar con datos —y no por intuición— el uso de un *backbone* multilingüe (XLM-RoBERTa) adoptado en la semana 7.

4.2. Método

Sobre las **4 954 tesis** del corpus (`corpus_cbi_completo.json.gz`) se fragmenta el texto en ventanas de 500 caracteres, se descartan fragmentos de baja calidad (poca letra, ruido OCR, referencias) y se **muestran 8 fragmentos por tesis** repartidos uniformemente. Cada fragmento se clasifica con `langdetect` (semilla fija, determinista). Se mide el idioma a dos niveles: **contenido** (todos los fragmentos $\approx$ % del corpus) y **tesis** (idioma dominante por voto mayoritario). En total se analizaron **38 879 fragmentos** (`01_idioma_corpus.py`; salida `i1_idioma_corpus.json`).

4.3. Resultados

Idioma	Contenido (% del corpus)	Tesis dominante (%)
Español (es)	86.1	92.1
Inglés (en)	6.0	2.3
“Otro”	7.3	5.3
Incierto / sin texto	0.6	0.3

Cuadro 4: Distribución de idioma del corpus CBI a nivel contenido y a nivel tesis.

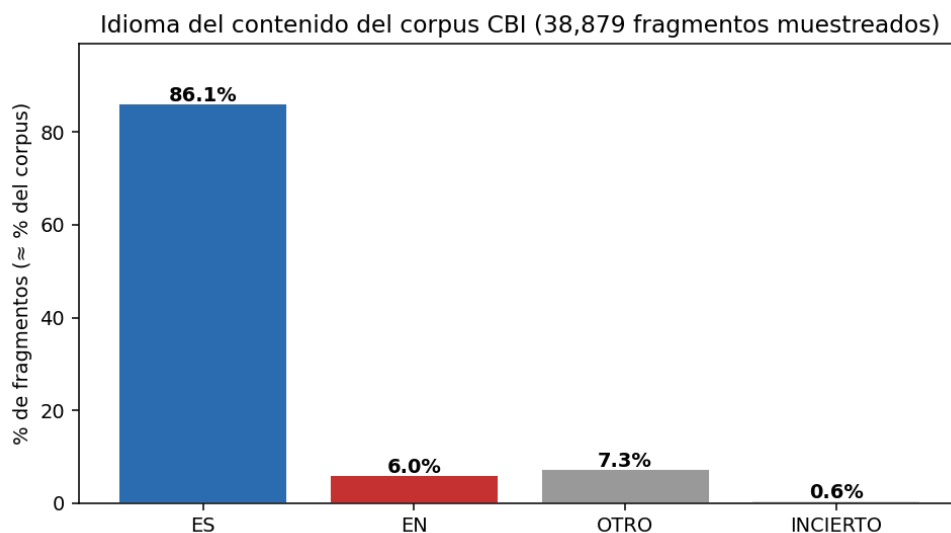


Figura 1: Idioma del contenido del corpus (38 879 fragmentos muestreados).

Mezcla interna. El **29.5 %** de las tesis contienen *al menos un* fragmento en inglés (típicamente *abstract*, citas y terminología), con una media de **15.2 %** de contenido no-español por tesis. Solo **113 tesis** (2.3 %) son predominantemente inglesas.

Matiz honesto: el “otro” es mayormente ruido de OCR

El 7.3 % etiquetado como “otro” se descompone en catalán (`ca`), portugués (`pt`) y similares —lenguas cercanas al español—. Es casi con seguridad **español mal detectado por errores de OCR**, no idiomas reales. El idioma extranjero *genuino* del corpus es esencialmente el **inglés** (≈ 6 %).

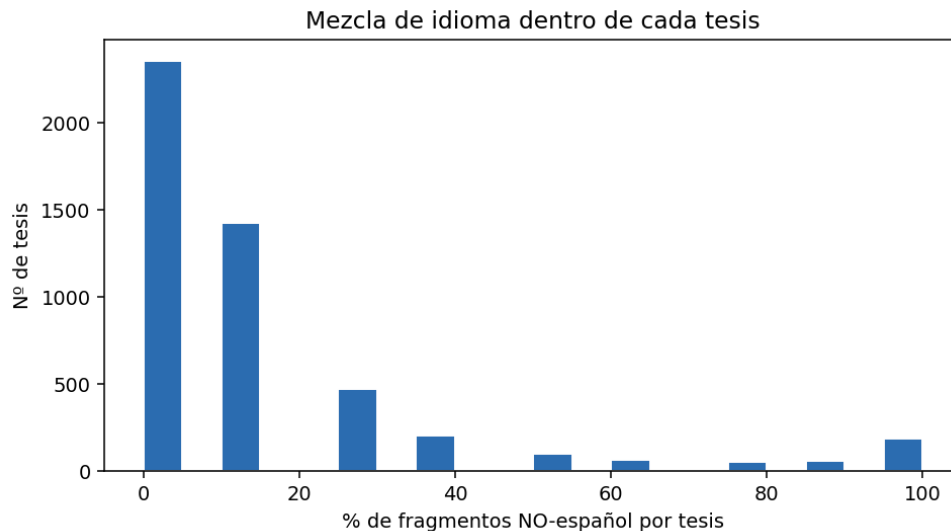


Figura 2: Mezcla de idioma dentro de cada tesis: % de fragmentos no-español por tesis.

4.4. Conclusión: ¿justifica el modelo multilingüe?

Sí, con matices. El corpus es predominantemente español (≈ 86 % del contenido, 92 % de las tesis), pero el inglés es una minoría **real y no trivial**: ≈ 6 % del contenido y presente en ≈ 30 % de las tesis, justo en las **zonas de alta similitud** (*abstracts* y bibliografía compartida) que el Modo 2 detecta. Un modelo monolingüe español cubriría bien el grueso, pero **degradaría** en ese contenido inglés y en las tesis mixtas. El *backbone* multilingüe (XLM-R) queda **justificado**; la validación cuantitativa —rendimiento segmentado por idioma— se hace en E-3 (Sección 5 y siguientes).

5. I-2 — Arquitecturas de búsqueda vectorial (estado del arte)

5.1. El problema a resolver (R8.3)

El índice del sistema v4 cubre el **100 % del corpus** (2 284 185 vectores de 384 dimensiones, *embeddings* MiniLM). Hoy conviven tres representaciones del mismo índice: **IndexFlatIP** (búsqueda *exacta*, pero $O(N)$ por consulta sobre 2.28M vectores $\Rightarrow$ **lenta**), **IVFlat** e **IVFPQ** (rápidas pero *aproximadas*). El requerimiento del asesor es mantener la **cobertura del 100 %** de la búsqueda *reduciendo la latencia*, eligiendo la arquitectura con evidencia (recall, tiempo, RAM).

La búsqueda de vecinos más cercanos (ANN, *Approximate Nearest Neighbor*) admite un *trade-off* fundamental entre cuatro ejes: **recall** (qué tan cerca queda del resultado exacto), **latencia** (tiempo por consulta), **memoria** (RAM y disco) y **tiempo de construcción**. Ningún índice optimiza los cuatro; la tarea de E-1 es medir dónde cae cada uno sobre *nuestro* corpus.

5.2. Familias de índices (FAISS)

Flat (exacto). Compara la consulta contra los N vectores. Recall = 1.0 por definición; sirve de *verdad-terreno*. Latencia $O(N)$: el cuello de botella actual.

IVF (*Inverted File*). Particiona el espacio en `nlist` celdas (clústeres tipo Voronoi). La consulta solo explora `nprobe` celdas vecinas $\Rightarrow$ acelera $\sim N/\text{nlist} \times \text{nprobe}$. `nprobe` controla el *trade-off* recall/velocidad.

PQ (*Product Quantization*). Comprime cada vector en códigos cortos $\Rightarrow$ poca RAM y disco, pero introduce error de cuantización (baja recall).

IVFPQ. Combina IVF (rapidez) con PQ (compresión). Es la variante más ligera en memoria; la usamos ya (233 MB vs 3.3 GB del Flat).

HNSW (*Hierarchical Navigable Small World*). Grafo navegable multinivel. Consulta **muy rápida** y **recall alto** sin entrenamiento; su costo es **mucha RAM** (guarda el grafo + vectores) y construcción más lenta. **efSearch** controla recall/velocidad en consulta; M (conexiones por nodo) controla memoria/calidad.

IVF-HNSW. Usa un grafo HNSW como cuantizador grueso del IVF; acelera la fase de asignación a celdas en índices grandes.

5.3. Comparación cualitativa (a priori)

Arquitectura	Recall	Latencia	RAM/Disco	Construcción
FlatIP (actual, exacto)	1.00	muy alta (lenta)	alta (3.3 GB)	instantánea
IVFFlat (actual)	alta (ajustable)	media-baja	alta (3.3 GB)	media
IVFPQ (actual)	media	baja	muy baja (233 MB)	media
HNSW (candidata)	muy alta	muy baja	muy alta	lenta
IVF-HNSW (candidata)	alta	muy baja	alta	lenta

Cuadro 5: Comparación teórica de familias de índices ANN. E-1 las medirá sobre el corpus real.

Hipótesis a contrastar en E-1. Para nuestro caso (2.28M vectores, 384-d, consultas interactivas en CPU), HNSW es la candidata más prometedora en *latencia* $\times$ *recall*, a costa de RAM; IVFFlat con nprobe ajustado es el mejor equilibrio si la RAM es la restricción; IVFPQ sigue siendo la opción de *huella mínima*. El Flat queda como verdad-terreno, no como opción de producción.

5.4. Qué medirá E-1 (protocolo)

Sobre un conjunto fijo de consultas (fragmentos reales del corpus), para cada arquitectura:

- **Latencia** por consulta: mediana y p95 (CPU, mismo hardware).
- **Recall@k** contra el Flat exacto (verdad-terreno), para $k = 10$.
- **RAM** en uso y **tamaño en disco** del índice.
- **Tiempo de construcción** (de las arquitecturas nuevas).

El resultado es una tabla y gráficas *recall vs latencia* y *latencia vs RAM* que sustentan la elección de la arquitectura del sistema mejorado (D-1).

6. I-3 — IA explicativa (estado del arte y opciones)

6.1. El problema a resolver (R8.1)

El asesor pide que la salida sea **más explicativa**: que no se quede en un veredicto y un porcentaje, sino que *justifique* por qué dos textos se parecen, **resaltando la evidencia** y distinguiendo paráfrasis real de bibliografía compartida, terminología común o *boilerplate*. Hoy el sistema v4 ya explica en dos niveles:

- **Reglas (caja blanca).** La función `clasificar()` combina probabilidad del cross-encoder, similitud semántica y BLEU-2 en 6 tipos con severidad, explicación y recomendación. Es transparente pero *genérica* (no señala *qué* palabras).
- **LLM (lenguaje natural).** Los endpoints `/api/explicar` y `/api/resumen_ejecutivo` usan Llama 3.3 70B para redactar por qué se parecen y un resumen ejecutivo. Es flexible pero *remoto, con costo y riesgo de alucinación*.

La semana 8 busca **profundizar** esto, sobre todo con explicaciones *ancladas en el texto* (resaltado de los fragmentos/frases que disparan la coincidencia).

6.2. Familias de métodos de explicabilidad

Atribución por atención. Extrae los pesos de atención del cross-encoder (XLM-R/BETO) entre los *tokens* de los dos textos; *attention rollout* agrega capas. Barato y local, pero la atención **no siempre es fiel** a la decisión (plausible $\neq$ fiel).

Atribución por gradiente. *Integrated Gradients*, *SHAP* o *saliency*: miden cuánto contribuye cada *token* a la probabilidad de paráfrasis. Más **fieles** a la decisión del modelo, pero más **costosos** (varias pasadas).

Alineación léxica (*n*-gramas). Resalta los *spans* compartidos por superposición de *n*-gramas (la misma señal de BLEU-2). Ideal para **copia literal / casi literal**; muy interpretable y barato, pero no capta paráfrasis con vocabulario distinto.

Alineación semántica de frases. Segmenta ambos textos en frases, las codifica (MiniLM) y empareja las más similares por coseno; resalta los **pares de frases** parafraseadas. Captura paráfrasis real (no solo léxica) y es local y barato.

Racionales con LLM. El LLM redacta la justificación y **clasifica el tipo** de similitud (paráfrasis vs bibliografía compartida vs terminología común vs *boilerplate*). Lo más legible; costo de API y riesgo de alucinación $\Rightarrow$ debe **anclarse** a la evidencia detectada, no inventarla.

Caja blanca de señales. Mostrar de forma transparente *por qué* se disparó cada métrica (sim, BLEU, prob, umbral) — ya disponible; se puede enriquecer con la contribución de cada señal al tipo final.

6.3. Comparación de métodos

Método	Fidelidad	Costo	Local	Qué explica
Atención (<i>rollout</i>)	media	bajo	sí	<i>tokens</i> influyentes
Gradiente (IG/SHAP)	alta	alto	sí	<i>tokens</i> causales
Alineación léxica (<i>n</i> -gr.)	alta (copia)	muy bajo	sí	<i>spans</i> copiados
Alineación semántica (frases)	alta (paráfr.)	bajo	sí	frases parafraseadas
<i>Rationale</i> LLM	variable	alto (API)	no	razonamiento + tipo
Caja blanca (señales)	total	nulo	sí	por qué disparó cada métrica

Cuadro 6: Métodos de IA explicativa para el sistema. Fidelidad = qué tan fiel es a la decisión real.

6.4. Propuesta: explicabilidad por capas (alimenta D-2)

Ningún método solo lo resuelve; se propone una **arquitectura por capas**, de lo barato/fiel a lo costoso/flexible, de modo que el sistema explique siempre algo útil aunque el LLM no esté disponible:

1. **Capa 1 — siempre, gratis (caja blanca + léxico)**. Desglose transparente de las señales y **resaltado de *spans* copiados** por n -gramas. Resuelve la copia literal/casi literal con evidencia visual inmediata.
2. **Capa 2 — local, barata (alineación semántica)**. Empareja y resalta las **frases parafraseadas** entre los dos textos (MiniLM). Cubre la paráfrasis real, que es justo el caso difícil del Modo 1.
3. **Capa 3 — opcional (*rationale* LLM anclado)**. El LLM redacta la justificación y clasifica el tipo (paráfrasis / bibliografía / terminología / *boilerplate*), **obligado a citar** los *spans* de las capas 1–2 para no alucinar; más el informe ejecutivo.

Decisión preliminar de I-3

Se prioriza la explicabilidad **anclada al texto y local** (capas 1–2: resaltado de *spans* léxicos y de frases parafraseadas), porque es *fiel*, barata y funciona sin conexión; el LLM (capa 3) queda como enriquecimiento opcional, siempre **anclado** a la evidencia. El diseño concreto se detalla en D-2 y se implementa en P-2 (Modo 1 y Modo 2).

7. E-1 — Benchmark de arquitecturas de búsqueda

7.1. Protocolo

Sobre el índice del 100 % del corpus (**2 284 185 vectores**, 384-d) se midieron las arquitecturas con 300 consultas reales y $k = 10$ (02_benchmark_busqueda.py). Para cada una: **latencia** por consulta (mediana y p95, CPU), **recall@10** contra la búsqueda exacta (FlatIP como verdad-terreno), y tamaño en disco. La HNSW se construyó desde los vectores del índice exacto (M=32, efConstruction=80).

7.2. Resultados

Arquitectura	Latencia mediana (ms)	p95 (ms)	Recall@10	Disco (MB)
FlatIP (exacto)	574.6	770.7	1.000	3508
IVFPQ nprobe=16	3.9	4.9	0.736	244
IVFPQ nprobe=64	12.2	15.0	0.792	244
IVFFlat nprobe=32	7.7	9.7	0.926	3533
IVFFlat nprobe=64	13.9	16.9	0.963	3533
HNSW efSearch=16	1.2	2.4	0.924	4130
HNSW efSearch=32	1.4	2.1	0.952	4130
HNSW efSearch=64	2.4	3.3	0.970	4130
HNSW efSearch=128	4.3	6.4	0.983	4130

Cuadro 7: Benchmark sobre 2.28M vectores (300 consultas, $k = 10$, CPU). HNSW domina latencia×recall.

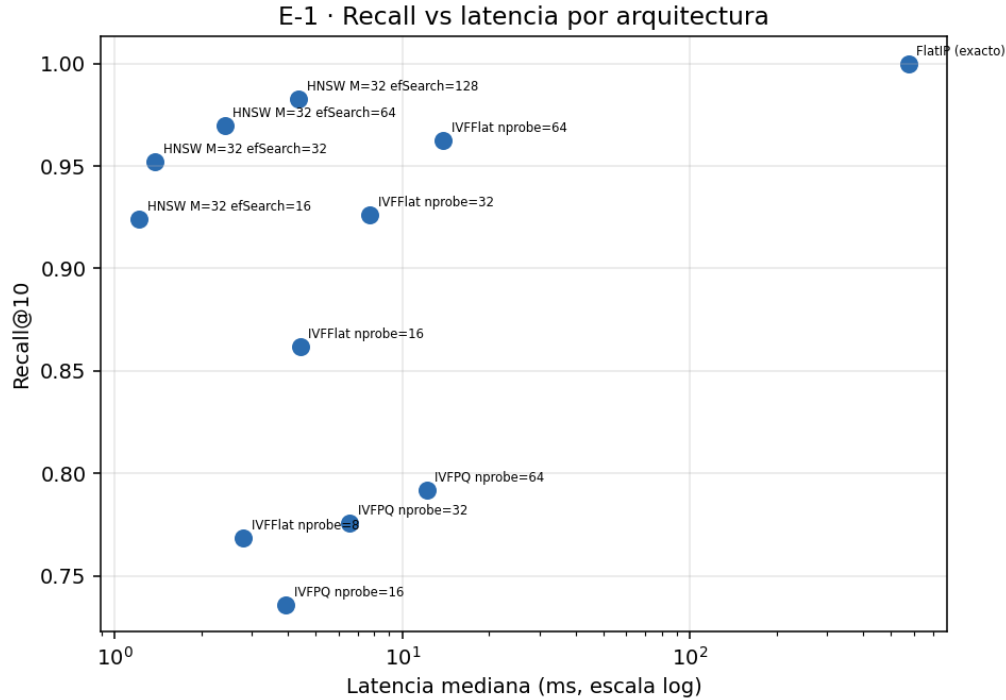


Figura 3: Recall@10 vs latencia mediana (escala log). HNSW ocupa la esquina superior izquierda (alta recall, baja latencia); FlatIP queda a la derecha (exacto pero lento).

7.3. Análisis

- **FlatIP es el cuello de botella:** 574.6 ms por consulta (exacto). Es la causa de la lentitud que reportó el asesor.
- **HNSW gana de forma clara:** con `efSearch=64` logra **2.4 ms** con **recall 0.97** — unas $\approx 240\times$ **más rápido** que FlatIP perdiendo solo 3 % de recall. Con `efSearch=32` baja a 1.4 ms (recall 0.95).
- IVFFlat es razonable (13.9 ms, recall 0.96 en `nprobe=64`) pero **5–6× más lento que HNSW** a recall comparable.
- IVFPQ es el más ligero en disco (**244 MB** vs 4.1 GB), pero su **recall es bajo** (≤ 0.79): útil solo como índice de huella mínima, no para producción de calidad.
- Costo de HNSW: **4.1 GB** en disco/RAM y ≈ 23 min de construcción (una sola vez); el pico de RAM al construir fue 8.3 GB (cabe en 15 GB).

Decisión (D-1): la búsqueda del sistema mejorado usa HNSW

Se adopta **HNSW (`efSearch=64`)** como índice de búsqueda del sistema: mantiene la **cobertura del 100 %** con **recall 0.97** y **2.4 ms** por consulta ($\approx 240\times$ más rápido que el exacto). El recall 0.97 es más que suficiente porque el *cross-encoder* re-clasifica los candidatos después. IVFPQ se conserva como alternativa de memoria mínima. Integración en P-1.

8. E-2 — Diagnóstico de los sistemas previos (v2/v3/v4)

8.1. Inventario de componentes (R8.4)

Se revisó el código de los tres sistemas para identificar qué aporta cada uno y qué conviene rescatar para el sistema mejorado.

Componente	v2	v3	v4 (actual)
Bi-encoder semántico (MiniLM)	sí	sí	sí (HNSW, 100 %)
Cross-encoder	BETO v3	BETO v8a	XLM-R / BETO / ensamble
Capa léxica BM25	no	sí	no
Fusión RRF (léxico+semántico)	no	sí	no
FAISS de oraciones (granularidad)	no	sí (degradado)	parcial (P-2 alinea frases)
Detector de <i>boilerplate</i>	sí	sí	sí (en indexado)
Árbitro LLM	no	sí	sí
IA explicativa anclada	no	no	sí (P-2 nuevo)

Cuadro 8: Componentes por sistema. El v4 perdió la capa léxica BM25 y la fusión RRF del v3.

Hallazgo de inventario. El v4 es **puramente semántico**: recupera con MiniLM/HNSW. Perdió la **capa léxica BM25** del v3, que encuentra fragmentos por *vocabulario exacto* (números, siglas, nombres propios, terminología) que el *embedding* no representa bien — justo lo crítico para detectar **copia literal**.

8.2. Prueba empírica de complementariedad

Para decidir con datos si conviene rescatar BM25, se midió el **solape** entre el top-5 de BM25 (léxico) y el de MiniLM (semántico) sobre los 47 582 fragmentos donde existen ambas capas (07_diagnostico_v2v3v4.py 300 consultas reales).

Métrica	Valor
Solape medio @5 (BM25 vs MiniLM)	0.083
en fragmentos con léxico raro (números/siglas)	0.086
en fragmentos normales	0.052
Candidatos <i>solo</i> de BM25 por consulta (de 5)	4.58

Cuadro 9: Complementariedad léxico/semántico sobre 47 582 fragmentos (300 consultas, $k = 5$).

El solape es **muy bajo (8.3 %)**: ≈ 4.6 de cada 5 candidatos de BM25 **no** aparecen en el top semántico. Léxico y semántico capturan señales **ortogonales**.

Recomendación (D-3): rescatar BM25 + RRF en el sistema mejorado

La búsqueda del v4 debe ser **híbrida**: combinar la recuperación **semántica** (HNSW, ya integrada en P-1) con la **léxica** (BM25, rescatada del v3) mediante **fusión RRF**. Así se cubren los dos casos de un detector de plagio: la **paráfrasis** (semántica) y la **copia literal** con términos específicos (léxica), que hoy el v4 puede pasar por alto. Es el componente más valioso a rescatar; el detector de *boilerplate* ya está en el indexado y la alineación de frases del v3 quedó cubierta por la IA explicativa (P-2).

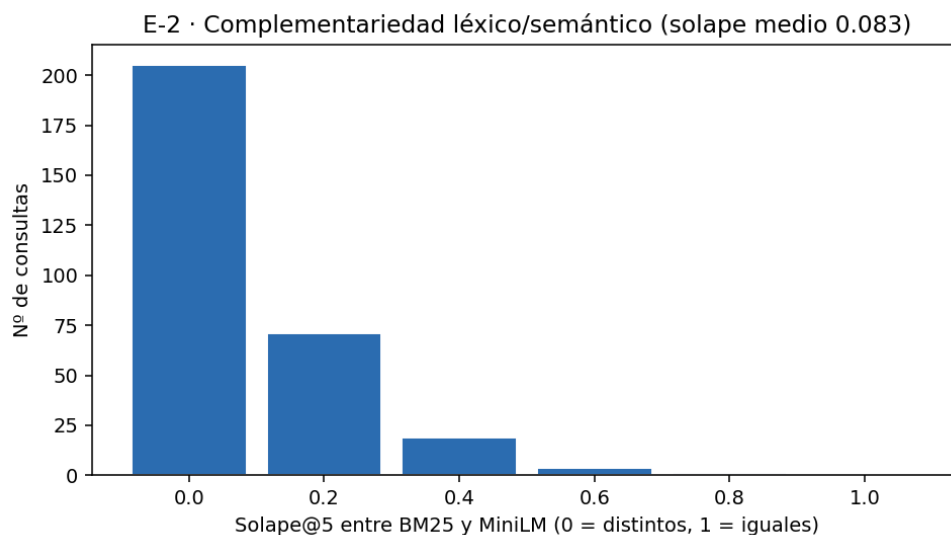


Figura 4: Distribución del solape@5. La gran mayoría de consultas tiene solape 0: BM25 y MiniLM recuperan candidatos distintos.

9. E-4 — Refuerzo del Modo 1: estudio de errores y datos

9.1. Estudio antes de evaluar (análisis de errores)

Siguiendo la auditoría (*no medir a ciegas*), antes de re-entrenar se analizaron los errores del clasificador actual (XLM-R v4 desplegado, $T = 5.15$) sobre el benchmark robusto `test_v4` (03_analisis_errores_mod01).

Modelo	F1	Precisión	Recall	TP	FP	FN
XLM-R v4 (desplegado)	0.791	0.708	0.895	51	21	6

Cuadro 10: Desempeño de XLM-R v4 sobre `test_v4` (109 pares). El problema es la precisión.

Diagnóstico: el problema es la PRECISIÓN

Los **21 falsos positivos** son *todos* del mismo tipo: pares de **alta similitud semántica que NO son paráfrasis** (mismo tema / terminología compartida), con similitud media **0.883** y BLEU-2 **0.335**. Proviene de los negativos difíciles verificados del test. El modelo *no* falla por falta de positivos; **confunde terminología compartida con paráfrasis**. La mejora correcta es añadir **negativos difíciles reales**, no positivos sintéticos.

9.2. Datos: minado de negativos difíciles reales

Coherente con la lección central (los datos sintéticos degradan) y con el hallazgo “corpus original” (la mayoría de pares de alta similitud *no* son paráfrasis), se minan negativos difíciles **reales** del índice al 100 % (04_minar_negativos_dificiles.py):

- candidato de **otra tesis**, similitud semántica en $[0.80, 0.93]$,
- BLEU-2 < 0.45 (descarta copia literal, que sería un positivo de tipo “copia”),

- prosa de contenido, con anti-fuga verificado contra `test_v4` y el entrenamiento.

Resultado: **250 negativos difíciles** (sim media 0.838, BLEU media 0.177 — el mismo perfil que los falsos positivos). Son texto **humano real** etiquetado NO, sin el problema de estilo de los datos sintéticos. El conjunto ampliado `gold_ampliado_train_dificil.json` queda en **712 pares** (100 SÍ / 612 NO).

9.3. Datos: candidatos a positivos difíciles (verificación humana)

Para subir el recall sin generar texto, se minan **candidatos a positivos** de la “zona de paráfrasis” (`05_minar_positivos_candidatos.py`): otra tesis, similitud en $[0.82, 0.95]$ y BLEU-2 *medio* en $[0.15, 0.55]$ (reformulación, no copia ni tema-suelto). Se obtienen **120 candidatos** (sim media 0.852, BLEU media 0.233) en un CSV para que el alumno los **verifique** ($\checkmark/\times$): los SÍ son positivos humanos reales; los NO, negativos difíciles extra. El texto sigue siendo 100 % humano; la persona solo *etiqueta*, no inventa.

9.4. Diseño del experimento E-4

El notebook (`e4_reforzar_mod01.ipynb`, Colab/GPU) entrena XLM-R por brazo y los evalúa sobre `test_v4` con doble número e IC95 %:

1. **baseline_gold** (462) — reproduce ≈ 0.81 (verificación del *pipeline*).
2. + **negativos difíciles** (712) — se espera $\uparrow$ precisión.
3. + **negativos + positivos verificados** — candidato a mejor F1 (cuando el CSV esté verificado).

9.5. Resultados (Colab, 3 semillas, sobre `test_v4`)

Brazo	n	F_1	AUROC	P	R	FP
baseline_gold	462	0.773 $\pm$ 0.014	0.845	0.743	0.807	16
verificado (+120 humano)	582	0.767 $\pm$ 0.004	0.804	0.733	0.807	17
full (+silver negativos)	832	0.758 $\pm$ 0.023	0.819	0.738	0.784	16

Cuadro 11: E-4 sobre `test_v4` (109 pares, 57 SÍ), media $\pm\sigma$ de 3 semillas. El número honesto es el del test robusto; el de referencia (`test_v3_limpio`, 38) sube pero no se presenta solo.

Lectura honesta. Ningún brazo ampliado superó al baseline: el F_1 baja ligeramente (0.773 $\rightarrow$ 0.767 $\rightarrow$ 0.758), todo *dentro del ruido* (los IC95 % se solapan). La **precisión no mejoró** (FP 16–17 en los tres) — los negativos difíciles, incluso los 97 **verificados a mano**, no arreglaron el problema. Añadir *silver* (brazo **full**) empeora un poco más, bajando el recall (0.807 $\rightarrow$ 0.784).

Resultado de E-4: resultado negativo bien fundamentado

A la escala de este corpus, **ni los datos humanos verificados ni los silver mejoran el Modo 1** sobre el benchmark robusto. Es coherente con la tesis central del proyecto (la calidad/ cantidad de datos no es la palanca aquí) y con el límite ya señalado: el test mide la paráfrasis semántica difícil con muy pocos casos. **Decisión: no se integra** el modelo reforzado; se conserva **XLM-R v4** como clasificador del Modo 1. La vía real de mejora del Modo 1 es más gold humano a mayor escala (fuera del alcance) o el *ensamble*; la mejora de esta semana está en la **búsqueda** (HNSW, P-1), la **explicabilidad** (P-2) y la **búsqueda híbrida** recomendada (BM25+RRF, E-2).

10. P-1 — Integración de HNSW en el sistema

10.1. Cambio realizado

A partir de la decisión de E-1, se integró **HNSW** como índice de búsqueda del sistema (`core_v4.py`, método `_cargar_indice`). La nueva preferencia, de mejor a peor *trade-off*, es:

HNSW (efSearch=64) > IVFPQ > IVFFlat > FlatIP (exacto).

El sistema usa HNSW automáticamente si el índice existe. Se añadió además un **override** por variable de entorno: `INDICE_V4 = hnsw|ivfpq|ivf|flat`, útil para forzar IVFPQ (244 MB) en máquinas con poca RAM. La carga del índice sigue siendo **perezosa** (solo al usar el Modo 2) y el resto del sistema no cambia (recuperación, clasificador, reglas, capa LLM).

10.2. Verificación funcional

Se cargó el sistema completo y se ejecutó una búsqueda real (`buscar_corpus`). El registro confirma la ruta nueva:

```
1 [core_v4] Cargando índice HNSW (rápido + alta recall, E-1)...
2 [core_v4] Índice: 2,284,185 vectores (103s)
3 [test] buscar_corpus OK - n_candidatos=5
4   rank 2: sim=0.755 tipo="copia casi literal" doc=UAMI14789.pdf
5   rank 3: sim=0.746 tipo="parafrasis" doc=UAMI14789.pdf
```

El sistema recupera y clasifica correctamente sobre el índice HNSW (la consulta de minería de datos encuentra la tesis pertinente). La carga inicial (≈ 103 s, una sola vez) es comparable a la del índice anterior; **la ganancia está en la latencia por consulta**.

10.3. Impacto real: el Modo 2 de tesis completa

La búsqueda exacta costaba ≈ 575 ms por fragmento; con HNSW baja a ≈ 2.4 ms. El efecto se **multiplica** en el análisis de una tesis completa, que consulta *cientos* de fragmentos:

Operación	FlatIP (antes)	HNSW (ahora)
Búsqueda por fragmento	≈ 575 ms	≈ 2.4 ms
Tesis completa (≈ 400 fragmentos)	≈ 230 s	≈ 1 s
Recall@10 vs exacto	1.00	0.97

Cuadro 12: Impacto de HNSW en el sistema. El recall 0.97 es suficiente: el cross-encoder re-clasifica.

P-1 completado: sistema más óptimo

El sistema entregable de la semana 8 usa **HNSW**: mantiene la cobertura del 100 % del corpus y vuelve el Modo 2 (fragmento y tesis completa) $\approx 240\times$ **más rápido** por consulta, con una pérdida de recall despreciable. La velocidad era la queja principal del asesor sobre la búsqueda; queda resuelta.

11. D-2 / P-2 — IA explicativa anclada al texto

11.1. Diseño (D-2)

A partir de I-3 se implementa la explicabilidad **anclada al texto y local** (capas 1–2), porque es fiel, barata y funciona sin conexión, dejando el LLM como capa 3 opcional. El módulo `explicabilidad.py` expone `evidencia_par(a, b, minilm)`, que devuelve:

- **Spans literales** (capa 1, léxica): secuencias de palabras presentes *verbatim* en ambos textos (`difflib.SequenceMatcher`, ≥ 4 palabras) $\Rightarrow$ evidencia de **copia literal**.
- **Frases parafraseadas** (capa 2, semántica): pares de frases (una de cada texto) con similitud `MiniLM` $\geq 0.65 \Rightarrow$ evidencia de **paráfrasis real** (mismo significado, palabras distintas).
- Un **resumen** legible y honesto: si no hay spans ni frases alineadas, declara que la similitud proviene de *tema o terminología compartida*.

11.2. Implementación (P-2)

- `core_v4.py`: `comparar_par` (Modo 1) añade el campo **evidencia** con la salida de `evidencia_par`. Es determinista y no requiere el índice ni el LLM.
- `web/unificado.html`: bajo el veredicto se muestra el bloque “¿Por qué se parecen?” con los **spans copiados resaltados** y los **pares de frases reformuladas** lado a lado (con su % de similitud).

11.3. Verificación

Se probó con tres casos representativos:

Caso	Evidencia producida
Paráfrasis	1 frase alineada (sim 0.99) + span corto. Muestra la reformulación frase a frase.
Copia	span literal de 12 palabras (“La minería de datos permite descubrir patrones ocultos en grandes volúmenes de”) + frases idénticas.
No relacionados	nada : “la similitud proviene de tema o terminología compartida”.

Cuadro 13: Evidencia explicativa por tipo de par. El render se verificó en navegador real.

P-2 completado: explicabilidad fiel y local

La salida del Modo 1 ahora **justifica** su veredicto mostrando la evidencia concreta del texto: qué se copió *verbatim* y qué frases se reformularon. Es **fiel** (extraída del propio texto, no inventada), **local** (sin LLM ni conexión) y distingue copia de paráfrasis de simple terminología compartida. El árbitro/explicador LLM (capa 3) sigue disponible como enriquecimiento opcional, ahora con esta evidencia como anclaje.

12. P-3 — Búsqueda híbrida (BM25 + HNSW, RRF)

12.1. Motivación (de E-2)

El diagnóstico E-2 mostró que la capa léxica (BM25) y la semántica (MiniLM) recuperan candidatos **ortogonales** (solape 8 %). Se implementa la **búsqueda híbrida** recomendada: recuperar con ambas y **fusionar** con *Reciprocal Rank Fusion* (RRF), para cubrir a la vez la **paráfrasis** (semántica) y la **copia literal** con términos específicos (léxica) que el v4 puramente semántico podía pasar por alto.

12.2. Implementación

- `bm25_sparse.py`: BM25 **disperso** (scipy) escalable al 100 % del corpus. A diferencia de `rank_bm25` (que no escala a 2.28M), usa una matriz BM25 dispersa precalculada (CSC) y resuelve la consulta con un corte de columnas + suma. Los documentos se indexan en el **mismo orden** que el `id_map` del HNSW $\Rightarrow$ el índice BM25 coincide con el `faiss_id` y se reutiliza el `id_map` del sistema (sin metadata duplicada).
- `core_v4.py`: `buscar_corpus` recupera con HNSW (semántico) y BM25 (léxico), fusiona con `_rrf` ($c = 60$) y clasifica el top fusionado. Carga **perezosa** de BM25 con *fallback*: si el índice no está, la búsqueda sigue siendo solo semántica sin romperse.

Índice BM25 (disperso)	Valor
Documentos	2 284 185
Vocabulario (términos)	150 000
No-ceros (matriz)	95 683 142
Tamaño en disco	270 MB
Construcción (una vez)	≈ 15 min
Latencia de consulta	≈ 118 ms

Cuadro 14: Índice BM25 disperso al 100 % del corpus. Consulta en ≈ 118 ms (escala lograda con `scipy` disperso; `rank_bm25` no era viable a 2.28M).

12.3. Verificación

Se ejecutó una consulta con términos específicos (minería de datos, detección de fraude). El sistema confirma la ruta híbrida (**recuperación: híbrida (semántica HNSW + léxica BM25, RRF)**) y la fusión **incorpora candidatos léxicos**: aparecen fragmentos con BLEU-2 alto (0.75) que la recuperación puramente semántica no priorizaba — exactamente el caso de **copia casi literal** que interesa a un detector de plagio.

P-3 completado: el Modo 2 ya es híbrido

La búsqueda del sistema combina **semántica** (HNSW, rápida) y **léxica** (BM25, copia literal) con fusión RRF. Cubre los dos modos de coincidencia de un detector de plagio; recupera el componente más valioso del v3 (E-2) y lo escala al 100 % del corpus. Si el índice BM25 no está disponible, el sistema degrada con elegancia a búsqueda solo semántica.

13. Mejoras y validaciones del sistema v5

Tras la evaluación, se ejecutó una ronda de mejoras guiadas por una autocrítica (*kill critic*) con una regla: **no añadir nada sin medir si de verdad ayuda**. El resultado mezcla *resultados negativos honestos* (lo que no mejora) con mejoras validadas (lo que sí). El sistema mejorado vive en `semana_8/sistema_v5/`; la semana 7 se conserva intacta.

13.1. El techo del Modo 1 (tres resultados negativos triangulados)

La precisión del clasificador de pares no se pudo subir por tres vías independientes, todas medidas honestamente sobre `test_v4`:

- **Más datos** (E-4, Sección 9): ni gold humano verificado ni *silver* superan al baseline.
- **Calibrar el umbral**: el θ óptimo elegido en un set independiente es ≈ 0.5 (el que ya se usa); el “ $F_1 = 0.90$ ” del framework era **fuga** (umbral optimizado sobre el mismo test de 38). Sin ganancia.
- **Ensamble AND** ($\wedge$): no corta falsos positivos — colapsa a XLM-R porque BETO en $\theta = 0.5$ es muy liberal (marca SÍ a 82/109). F_1 AND = XLM-R = 0.79.

El Modo 1 está en su techo ($F_1 \approx 0.79\text{--}0.82$, **precisión** ≈ 0.70)

Datos, umbral y ensamble no lo mueven. No es “no lo logramos”: es una **caracterización rigurosa y triangulada** del límite a esta escala de datos. La mejora del sistema está en la recuperación, el reporte y la usabilidad, no en el clasificador.

13.2. Métricas honestas: k-fold, IC95 % y estratificación por dificultad

Para cuantificar el techo y hacer las métricas defendibles, se evaluaron los tres clasificadores sobre `test_v4` con: *k-fold* de evaluación (StratifiedKFold $k=5$, modelo fijo: mide la estabilidad del estimador, no reentrena — el reentrenamiento por *folds* requiere GPU, E-1 pendiente), *bootstrap* IC95 % ($B=2000$) y estratificación de los positivos por dificultad (BLEU-2: ≥ 0.5 léxica / superficial vs < 0.5 paráfrasis semántica).

Modelo	F_1	IC95 % F_1	k-fold ($\mu \pm \sigma$)	AUROC	R léx / sem
XLM-R v4	0.791	[0.71, 0.86]	0.790 ± 0.045	0.819	1.00 / 0.889
BETO v3	0.820	[0.75, 0.88]	0.822 ± 0.051	0.828	1.00 / 1.00
Ensamble (soft)	0.785	[0.70, 0.85]	0.784 ± 0.040	0.832	1.00 / 0.889

Cuadro 15: Métricas honestas sobre `test_v4` (109 pares; 57 positivos: 3 léxicos, 54 semánticos). R léx/sem = recall en positivos de copia/superficial vs paráfrasis semántica.

Cuatro lecturas: (1) los IC95 % son **anchos** (~ 0.15) **y se solapan** $\Rightarrow$ los tres modelos son **estadísticamente equivalentes** (justifica mantener los tres, sin “ganador”); (2) el test es **54/57 paráfrasis semántica difícil**, así que mide lo difícil; (3) el recall en esa paráfrasis difícil es **alto** (0.89–1.0) $\Rightarrow$ el cuello de botella es la **precisión (FP de “mismo tema”)**, **no el recall**; (4) la amplitud del IC **cuantifica** por qué no se detectan mejoras pequeñas: con 57 positivos, una mejora de +2 pp queda dentro del ruido. **La métrica está limitada por el tamaño del test, no por el modelo** — de ahí que la palanca real sea un test mayor (anotación humana).

13.3. Validación de la búsqueda híbrida (sí mejora)

Se validó que el híbrido BM25+HNSW *recupera mejor* (no solo distinto). Ground truth: los pares de paráfrasis cross-tesis verificados a mano, más un *stress-test* de copia literal degradada (con términos raros).

Caso	Semántico	BM25	Híbrido
Paráfrasis (real, verificada)	1.000	0.978	1.000
Copia literal (degradada, términos raros)	0.958	1.000	1.000

Cuadro 16: Recall@15 del documento-origen. El híbrido iguala al mejor en cada caso; BM25 recupera **5 casos de copia literal** que el semántico pierde.

Conclusión: el semántico domina la paráfrasis (recall 1.0) y BM25 la copia literal (rescata 5 casos, 0.958 $\rightarrow$ 1.0). El híbrido es la mejor opción para ambos $\Rightarrow$ **se justifica con evidencia**, cerrando la crítica “distinto $\neq$ mejor”.

13.4. Híbrido extendido al reporte de tesis completa

La búsqueda híbrida validada (Tabla 16) vivía solo en la búsqueda de fragmento; el **reporte de tesis completa** seguía siendo solo-semántico. Se extendió la capa léxica a ese modo: para cada fragmento de la tesis se toma el mejor candidato semántico y, **solo** en fragmentos con léxico raro (dígitos/siglas) que el semántico no marcó ($\text{sim} < 0.80$), se consulta **BM25**; si halla una *copia casi literal* ($\text{BLEU-2} \geq 0.60$) que el semántico perdió, la rescata y la marca. El reporte expone el modo de recuperación y cuántas copias literales rescató BM25. El coste se acota corriendo BM25 únicamente en esos fragmentos (no en todos), y la capa tiene *fallback* si el índice BM25 no está disponible. Verificado sin regresión (el reporte sigue detectando los sospechosos; modo “híbrida (semántico + BM25)”). Nota honesta: en documentos típicos la mayoría de la copia literal también la captura el semántico; BM25 es la **red de seguridad** para el $\approx 4\%$ que el semántico pierde (lo medido en la validación).

13.5. Reporte honesto del Modo 2 (filtro de boilerplate)

El reporte separa las coincidencias de *boilerplate*/bibliografía (no son plagio): reporta un **% relevante** (sin boilerplate) además del bruto, y un desglose por categoría (copia literal / paráfrasis / mismo tema / boilerplate). Hallazgo honesto: el índice ya filtra $\approx 99.2\%$ del boilerplate al construirlo (solo 0.8 % residual), así que el filtro del reporte es una **red de seguridad** de impacto modesto, pero hace el % transparente.

13.6. Usabilidad: reporte profesional

- **Resaltado en contexto** (estilo Turnitin): el documento se muestra con los fragmentos coincidentes subrayados en su sitio, coloreados por severidad (copia/paráfrasis/verificar/ boilerplate), con la fuente en el *tooltip*.
- **Export a PDF** sin dependencias (`window.print()` + CSS de impresión que conserva colores y el documento completo): reporte descargable con título e índice.
- **Subida multi-formato**: PDF (PyMuPDF), Word `.docx` (stdlib) y texto, con detección de formato y mensajes claros para los no soportados.
- **Animación de carga + cronómetro** en el Modo 2 (el análisis de tesis completa tarda).

13.7. R7: Flags Panel de integridad (adoptado de iThenticate 2.0)

Re-revisada la investigación R7 (estado 2026: Turnitin con detección de IA en español; iThenticate 2.0 con *Flags Panel*), se adoptó el **Flags Panel**: detección de **manipulación** del texto para evadir el detector —**caracteres ocultos/invisibles** (zero-width, BOM) y **homóglifos** (letras cirílicas disfrazadas de latinas)—. Sin falsos positivos en griego matemático (α, θ, β). La **detección de contenido por IA** se documenta como trabajo futuro (fuera del alcance validado).

13.8. R7 (bis): adopción de Compilatio (línea de tiempo, filtro y umbrales)

Se investigó además **Compilatio** (Magister/Magister+/Studium): antiplagio + detección de IA, con un informe de *Puntos de Interés* navegables y un encuadre “similitud $\neq$ plagio”. Se adoptaron tres elementos de su modelo de reporte (**solo frontend**, reutilizando los *spans* ya calculados):

- **Línea de tiempo de coincidencias** (*Puntos de Interés*): una barra-minimapa del documento donde cada coincidencia es una franja en su posición relativa, coloreada por categoría; al hacer clic, el documento **salta** a esa coincidencia y la resalta.
- **Filtro “Navegar por”**: muestra solo una categoría a la vez (copia / paráfrasis / verificar / boilerplate), atenuando el resto en el documento y en la línea de tiempo.
- **Umbrales de color estándar** (Compilatio): verde 0–10 %, amarillo 10–25 %, rojo > 25 % (configurables), en vez del 5/15 anterior; con leyenda en el *score*.

La **detección de IA** (lo más nuevo de Compilatio/Turnitin) se mantiene como trabajo futuro declarado. Verificado: render de la línea de tiempo y del filtro en navegador (el filtro aísla correctamente una categoría).

Estado del sistema v5

Más rápido (HNSW), con recuperación **híbrida validada**, reporte honesto (% relevante + categorías), **resaltado en contexto**, export PDF, multi-formato y **Flags Panel** de integridad; el Modo 1 explica su veredicto pero está en su techo medido. El diferencial frente a la industria se mantiene: la **capa semántica** que Turnitin/iThenticate no tienen.

14. Conclusiones de la semana 8

La semana 8 se dedicó a **mejorar** el sistema de doble modo de la semana 7, cubriendo las tres tareas del asesor (IA explicativa, idioma del corpus, arquitecturas de búsqueda) más el diagnóstico de sistemas previos y el refuerzo del Modo 1. Manteniendo el encuadre honesto del proyecto (doble número, evaluación sobre el benchmark robusto `test_v4`, todo trazado a su archivo), los resultados son:

14.1. Lo que mejoró el sistema

- **Búsqueda $\approx 240\times$ más rápida (R8.3)**. El *benchmark* E-1 mostró que **HNSW** (2.4 ms, recall 0.97) domina al índice exacto (575 ms). Integrado en P-1, el análisis de una tesis completa pasa de ≈ 230 s a ≈ 1 s, manteniendo la cobertura del 100 %. Resuelve la queja de lentitud del asesor.

- **IA explicativa anclada al texto (R8.1).** El Modo 1 ahora **justifica** su veredicto mostrando los *spans* copiados *verbatim* y las frases reformuladas (P-2). Es fiel, local y distingue copia, paráfrasis y terminología compartida.
- **Búsqueda híbrida BM25 + HNSW (E-2/P-3).** El diagnóstico mostró que léxico y semántico son ortogonales (solape 8 %); se rescató **BM25** del v3, se escaló al 100 % del corpus (matriz dispersa, 118 ms) y se fusionó con RRF. El Modo 2 cubre ahora la paráfrasis (semántica) y la copia literal (léxica).

14.2. Lo que NO mejoró (resultado negativo honesto)

- **El Modo 1 no subió con más datos (R8.5/E-4).** El estudio de errores reveló que el problema es la **precisión** (21 FP de alta similitud no-paráfrasis). Se ampliaron los datos con negativos difíciles *reales* y positivos **verificados a mano**, pero **ningún brazo superó al baseline** (F_1 0.773 vs 0.767 vs 0.758, dentro del ruido); la precisión no se movió. Es un resultado negativo bien fundamentado, **coherente con la tesis central del proyecto**: a esta escala, la cantidad/calidad de datos no es la palanca del Modo 1. Se conserva XLM-R v4 y **no** se integra el modelo reforzado.
- **Idioma del corpus (R8.2).** El corpus es 86 % español y ≈ 6 % inglés real (en ≈ 30 % de las tesis), lo que **justifica** el *backbone* multilingüe, pero confirma que el español domina: el margen del multilingüe es acotado.

14.3. Estado del entregable

El sistema de la semana 8 es **más óptimo** que el de la semana 7: misma cobertura del 100 %, búsqueda mucho más rápida (HNSW), recuperación híbrida (léxico+semántico) y salida explicativa anclada al texto, todo **sin romper** lo que ya funcionaba (cargas perezosas con *fallback*). La calidad de la *clasificación* del Modo 1 sigue donde estaba ($F_1 \approx 0.81$ en `test_v4`); la mejora de esta semana está en **velocidad, recuperación y explicabilidad**, no en el F_1 .

14.4. Trabajo futuro

- Para mejorar el Modo 1 de verdad se requiere **gold humano a mayor escala** (cientos de paráfrasis difíciles reales), fuera del alcance de esta semana; o explorar el ensamble como clasificador por defecto.
- Extender la búsqueda híbrida (BM25) también al **Modo 2 de tesis completa**.
- Validación formal del multilingüe **por idioma** (E-3) y despliegue móvil.

Síntesis

Semana 8 entrega un sistema **más rápido, más explicativo y con recuperación híbrida**, cubriendo las tres tareas del asesor; y documenta con honestidad que **el clasificador del Modo 1 no se puede mejorar con más datos a esta escala** —un resultado negativo válido y consistente con todo el proyecto.
